



А можно ли добавить кастомные правила для своих типов файлов

Да, можно: это самый правильный следующий шаг — вынести правила в конфиг и дать системе сначала проверять пользовательские правила, а уже потом применять встроенные эвристики. ^[1] ^[2]

Для этого удобно использовать JSON или YAML-конфиг с правилами по расширениям, именам файлов, маскам путей и именам директорий, потому что Python штатно хорошо работает и с путями через `pathlib`, и с JSON-конфигурацией. ^[1] ^[2]

Как это добавить

Логика должна быть такой:

1. Проверить кастомные правила пользователя. ^[3]
2. Если ничего не совпало, применить встроенные правила по расширению и директории. ^[3]
3. Если и там нет совпадения, использовать fallback-стиль. ^[3]

Это удобно, потому что тогда ты сможешь отдельно задать, например, что `.qnn`, `.chem`, `.brain`, `.mesh`, `.ont` или любые свои форматы относятся к нужному smell-style. ^[3]

Пример конфига

Вот простой вариант `smell_rules.json`:

```
{
  "extensions": {
    ".qnn": "paint_timber",
    ".chem": "paint_timber",
    ".mesh": "paint_metal",
    ".scene": "paint_house"
  },
  "filenames": {
    "Dockerfile": "paint_metal",
    "Makefile": "paint_metal"
  },
  "directories": {
    "quantum": "paint_timber",
    "ontology": "paint_house",
    "cuda": "paint_metal"
  },
  "path_contains": {
    "experiments/brain": "paint_timber",
```

```
    "docs/render": "paint_house"  
  }  
}
```

Такой формат легко расширять и версионировать вместе с репозиторием. ^[2]

Как встроить в код

В `RepositorySmellPainter` можно добавить загрузку конфига и приоритет пользовательских правил:

```
import json  
from pathlib import Path  
  
class RepositorySmellPainter:  
    def __init__(self, smell_library=None, custom_rules_path=None):  
        self.smell_library = smell_library or DEFAULT_SMELL_LIBRARY  
        self.custom_rules = {  
            "extensions": {},  
            "filenames": {},  
            "directories": {},  
            "path_contains": {}  
        }  
        if custom_rules_path:  
            self.custom_rules.update(  
                json.loads(Path(custom_rules_path).read_text(encoding="utf-8"))  
            )  
  
    def classify_file(self, path: Path):  
        rel = path.as_posix()  
        suffix = path.suffix.lower()  
        filename = path.name  
        parent_names = {p.name.lower() for p in path.parents}  
  
        if suffix in self.custom_rules["extensions"]:  
            return self.custom_rules["extensions"][suffix], f"custom_extension:{suffix}"  
  
        if filename in self.custom_rules["filenames"]:  
            return self.custom_rules["filenames"][filename], f"custom_filename:{filename}"  
  
        for dirname, style in self.custom_rules["directories"].items():  
            if dirname.lower() in parent_names:  
                return style, f"custom_directory:{dirname}"  
  
        for fragment, style in self.custom_rules["path_contains"].items():  
            if fragment in rel:  
                return style, f"custom_path:{fragment}"  
  
        for dirname, style in DIRECTORY_RULES.items():  
            if dirname in parent_names:  
                return style, f"directory:{dirname}"  
  
        if suffix in EXTENSION_RULES:  
            return EXTENSION_RULES[suffix], f"extension:{suffix}"
```

```
return "paint_house", "fallback:generic"
```

Такой подход дает тебе расширяемый rule engine без тяжелой внешней зависимости, а если потом захочешь более сложные логические условия, можно уже перейти к полноценному rules-engine стилю. [4] [5]

Практически для тебя

Для твоих задач это особенно полезно, потому что у тебя могут быть нестандартные научные типы файлов, которых обычная эвристика не знает: симуляционные форматы, геометрия, квантовые схемы, нейромодели, онтологии, конфиги экспериментов. [3] Тогда репозиторий будет окрашиваться уже не “в среднем по больнице”, а по твоей собственной научной таксономии файлов и модулей. [3]

Что я могу сделать дальше

Я могу сразу подготовить версию скрипта с поддержкой `smell_rules.json`, приоритетом кастомных правил и примером под твои научные расширения. [3]

Сделать тебе готовый обновленный файл?

✱✱

1. <https://docs.python.org/3/library/pathlib.html>
2. <https://docs.python.org/3/library/json.html>
3. <https://www.airwick.ru/interesnoe/ustranenie-zapaxa/kak-izbavitsya-ot-zapaha-kraski-v-kvartire>
4. https://zerosteiner.github.io/rule-engine/getting_started.html
5. <https://pypi.org/project/rule-engine/>
6. <https://github.com/trialandsuccess/configurablejson>
7. <https://realpython.com/python-pathlib/>
8. <https://stackoverflow.com/questions/49380572/adding-another-suffix-to-a-path-that-already-has-a-suffix-with-pathlib>
9. <https://stackoverflow.com/questions/19078170/how-would-you-save-a-simple-settings-configuration-file-in-python>
10. <https://github.com/python/cpython/issues/121347>
11. <https://github.com/saurabh0719/py-rules-engine>